



PROJETO 4 – PROJETO DE UM S.O. (B.OS - BASIC OPERATING SYSTEM FOR TESTING AND THE APPRENTICESHIP OF OPERATING SYSTEMS)

OBJETIVOS

- I/O: teclado.
- Tratamento de interrupção
- Criação de um *shell* muito simples.

INFORMAÇÃO ADMINISTRATIVA

Este projeto pode ser realizado em grupos de no máximo 3 alunos. Um relatório descritivo dos procedimentos realizados e as respostas às questões presentes neste projeto devem ser entregues usando o modelo de relatório técnico-científico disponível no site da disciplina. **O relatório deve ser entregue impresso em duas vias para o professor da disciplina** até o dia 27/11/2018, no horário da aula.

Obs. Caso os alunos tenham interesse, é possível implementar este projeto também no windows utilizando uma estrutura de *cross compiler/cross linker* para a geração de código ELF. Para tanto, basta adquirir os seguintes pacotes: cygwin (com o gcc e o ld instalados) + NASM (for windows) + Qemu (for windows). Este tópico não será coberto durante este curso mas fica como dica para os alunos, caso desejem buscar este caminho.

PRELÚDIO: PRELIMINARES

Inicialmente, abra o arquivo *build.sh*. Adicione o seguinte parâmetro à linha do gcc: *-ffreestanding* (ao final da linha). Essa *flag* evita que o compilador use qualquer função oferecida pelo sistema hospedeiro, a não ser aquelas que definirmos durante a prática. Uma sugestão para a melhoria da função *print()* em *screen.h* é a criação de uma variável *length* do tipo *uint8* para armazenar o valor de *strlen(ch)+1*. Com isso, evita-se que a cada execução do laço *for* se chame novamente a função *strlen()* para que essa obtenha o mesmo valor.

TAREFA 1: IMPLEMENTAÇÃO DE E/S PARA O TECLADO (25/10/2018)

Para a implementação de um *shell* simples, teremos que criar a primeira função de entrada de dados, similar a função *scanf()* ou *gets()* em C. A função a ser criada se chama *readStr()* pra a leitura de uma *string*. Além desta função, também será criada uma função acessório chamada

strEql() a qual irá comparar duas *strings*.

A função *strEql()*, implementada na biblioteca *string.h*, é mostrada no Código 1. **Descreva o funcionamento desta função.**

```
uint8 strEql(string ch1, string ch2)
{
    uint8 result = 1;
    uint8 size = strlen(ch1);

    if (size != strlen(ch2))
        result = 0;
    else
    {
        uint8 i = 0;
        for(i; i <= size; i++)
            if (ch1[i] != ch2[i])
                result = 0;
    }

    return result;
}
```

Código 1: Código para a função *strEql()*.

Para a criação da função *readStr()*, esta deve ser inserida dentro do arquivo *kb.h*, no diretório *include* (Código 2). O objetivo desta função é traduzir os códigos de rastreamento de teclado (*scancode*) para código American Standard Code for Information Interchange (ASCII). Essa conversão é realizada no *switch()* que aparece no Código 2. Uma pequena amostra de como essa conversão deve ser realizada é apresentada dentro do *switch()*, para o *scancode* 1 (representando a tecla ESC). Para a tabela de *scancodes* completa consulte <http://appplant.tistory.com/11> e <https://www.millisecond.com/support/docs/v5/html/language/scancodes.htm>. Você deve criar as entradas de conversão até a entrada 57 da tabela de *scancodes*. O *scancode* fornecido é para teclados americanos. Para outros idiomas, consulte o site da Microsoft. Além disso, perceba que a função de leitura é um “laço infinito”. Pense qual será a condição de parada para este laço. **O código resultante deve ser comentado no relatório, incluindo como foi definida a condição de parada desta função.**

Vale observar que, para ler dados do teclado, é necessário ler dados de duas portas diferentes: 0x64 e 0x60. O endereço 0x64 permite ler o *status* do teclado, para saber quando uma tecla é pressionada. O endereço 0x60 permite a leitura do “valor” da tecla pressionada. *inportb(0x64) & 0x1* indica se uma tecla foi pressionada. Esta operação *bitwise* resultará em 1 para qualquer *scancode* válido (não existe *scancode* com valor 0). *inportb(0x60)* fornece o valor do *scancode* propriamente dito, o qual será convertido para código ASCII.

```

#ifndef KB_H
#define KB_H
#include "screen.h"
#include "system.h"
#include "types.h"

string readStr()
{
    char buff[256];
    string buffstr = buff;
    uint8 i = 0;
    uint8 reading = 1;

    while(reading)
    {
        if(inportb(0x64) & 0x1)
        {
            switch(inportb(0x60))
            {
                /*case 1:
                    printch((char)27);
                    buffstr[i] =(char)27;
                    i++;
                    break;*/

                case 2:
                    ...

            }
        }
        buffstr[i] = 0;
        return buffstr;
    }
}
#endif

```

Código 2: Código parcial para a função readStr().

TAREFA 2: IMPLEMENTAÇÃO DE INTERRUPÇÕES (30/10/2018 e 01/11/2018)

Iremos tratar nesta tarefa da Interrupt Description Table (IDT). Primeiramente, uma interrupção é um sinal enviado ao processador, a partir do hardware (ex. teclado, mouse, clock, disco etc) ou do próprio processador, no formato de uma exceção (*exception*). Esse sinal notifica ao processador, para que este pare o que está fazendo no momento, de modo a fazer uma outra atividade mais importante. Uma vez que o processador termina essa ação, o processador pode voltar eventualmente a desempenhar a ação interrompida, a partir de onde parou. Este sinal é chamado de “interrupção”, pois este literalmente interrompe o processador em sua execução corrente.

Por exemplo, se você estivesse assistindo um filme, e um colega seu o interrompesse perguntando o que você está fazendo, você pararia a ação de assistir o video e responderia seu colega, voltando a assistir o filme posteriormente. Este colega o interrompeu, basicamente pedindo para que você o respondesse, e você, por sua vez, tratou a solicitação do seu colega (*signal handling*), respondendo-o. Essa analogia também vale para o processador.

Voltando ao contexto da Tarefa 1, se você analisar o código para o tratamento de teclado,

você notará que, para conseguir a entrada do teclado, o processador continua em um laço, aguardando o acionamento de uma nova tecla, para que retorne de outra função. Nesse momento, a CPU não se importa se algo mais irá acontecer (ex. um *click* do mouse, um *clock tick*, uma exceção etc). Na prática, a CPU desconhece outras condições em nosso cenário. E isso ocorre porque ainda nada foi implementado para informar a CPU. É para isso que existem os sinais de interrupção, para informar a CPU sobre eventos externos. Outro exemplo ilustrativo pode ser obtido a partir de linguagens de programação como C#, Visual Basic, Java, Javascript etc. Se você já entrou em contato com alguma dessas, o conceito de evento, ou ainda, tratadores de evento (*event handlers*) provavelmente foi estudado. Esse é um conceito similar a teoria que estamos discutindo.

Com o intuito de tirar vantagem da implementação de interrupção, primeiramente, iremos informar ao processador sobre os processos. Faremos isso criando um vetor de 256 elementos, cada um contendo a informação das interrupções a serem implementadas. Não importa onde esse vetor será criado, o mais importante é que o computador saiba onde esse vetor está localizado na memória. Assim, após definir o vetor, informamos ao processador onde essa variável irá existir. Também precisamos informar ao computador qual é o seu tamanho na memória. Isso será realizado por meio da instrução assembly `lidt`, o que significa Load Interrupt Description Table. Vale ressaltar que a Interrupt Description Table (ou IDT) é na prática o vetor que foi citado anteriormente. Com isso, o processador estaria apto a acessar o conteúdo do vetor. De um modo simplificado: 1) cria-se o vetor; 2) define-se cada elemento do vetor; 3) salvamos o endereço do primeiro elemento do vetor e o seu tamanho em outro objeto e enviamos este objeto para a CPU pela chamada da instrução assembly `lidt`.

Para esta tarefa, as mudanças no SO não serão perceptíveis. As funções são implementadas na estrutura do sistema. Nesse contexto, criaremos os arquivos *util.h*, *util.c*, *idt.h*, *idt.c* e *isr.h* e *isr.c*. Além disso, serão criadas duas novas funções *inline* no arquivo *types.h*. Começaremos com a especificação dos arquivos *util.h* e *util.c*. O arquivo para *util.h* é apresentado no Código 3. Neste, são criadas três funções: *memory_copy()*, *memory_set* e *int_to_ascii()*.

```
#ifndef UTIL_H
#define UTIL_H

#include "types.h"

void memory_copy(char *source, char *dest, int nbytes);
void memory_set(uint8 *dest, uint8 val, uint32 len);
void int_to_ascii(int n, char str[]);

#endif
```

Código 3: Código para *util.h*.

A função *memory_copy()* pega os *nbytes* caracteres da origem e copia para o destino. *memory_set()* irá ajustar um valor específico *val* para o destino, para as *len* posições no vetor (mesmo valor ajustado *len* vezes). Finalmente, a função *int_to_ascii()* pega um valor inteiro e o transforma em uma *string*. **O objetivo desta tarefa é o aluno criar cada um desses códigos conforme instruído** (vide template no Código 4). Na função de conversão de inteiro para *string*, considere o número fornecido em módulo.

```
#include "util.h"

void memory_copy(char *source, char *dest, int nbytes) {
    ...
}

void memory_set(uint8 *dest, uint8 val, uint32 len) {
    ...
}

void int_to_ascii(int n, char str[]) {
    ...
}
```

Código 4: Template para *util.c*.

Em *types.h*, as funções *low_16()* e *high_16()* são definidas conforme mostrado no Código 5. Essas funções recebem um inteiro de 32 bits e o dividem. A função *low_16()* pega os 16 bits da direita de um inteiro de 32 bits (bits de ordem mais baixa). A função *high_16()* pega os 16 bits da esquerda de um inteiro de 32 bits (bits de ordem mais alta).

```
#ifndef TYPES_H
#define TYPES_H

...

typedef char* string;

#define low_16(address) (uint16)((address) & 0xFFFF)
#define high_16(address) (uint16)((address) >> 16 & 0xFFFF)

#endif
```

Código 5: Inserção de *low_16* e *high_16* em *types.h*.

Após a especificação de *util.h* e *util.c*, iniciaremos a especificação de *idt.h* e *idt.c*. O Código 6 apresenta a especificação de *idt.h*. *KERNEL_CS* refere-se ao seletor de segmento (no caso, o segmento do kernel). Além disso, definimos as estruturas *idt_gate_t* e *idt_register_t*. A primeira estrutura define cada elemento da Interrupt Descriptor Table ou Tabela de Descrição de

Interrupção (IDT), a qual contém 256 elementos. Cada byte de cada elemento possui um significado e esse mapeamento em estrutura evita que se tenha que usar instruções de baixo nível para se ter acesso a esses bytes individualmente. O somatório de todos os elementos da estrutura possui 64 bits, o que é um fato importante para evitar que o compilador adicione espaço ou código adicional. Por essa razão, adiciona-se o atributo `__attribute__((packed))`, o qual informa ao compilador para que ele gere o espaço ao qual se refere (no caso, a estrutura) da maneira como aparece no código. O compilador não deve adicionar nada por si mesmo (ex. por otimização de código). Após criar essa estrutura, cria-se um vetor de 256 elementos.

```
#ifndef IDT_H
#define IDT_H

#include "types.h"

#define KERNEL_CS 0x08

typedef struct {
    uint16 low_offset;
    uint16 sel;
    uint8 always0;
    uint8 flags;
    uint16 high_offset;
} __attribute__((packed)) idt_gate_t;

typedef struct {
    uint16 limit;
    uint32 base;
} __attribute__((packed)) idt_register_t;

#define IDT_ENTRIES 256
idt_gate_t idt[IDT_ENTRIES];
idt_register_t idt_reg;

/* Funções implementadas em idt.c */

void set_idt_gate(int n, uint32 handler);
void set_idt();

#endif
```

Código 6: Especificação para *idt.h*.

No Código 7 podemos ver a implementação das funções presentes em *idt.h* (Código 6). A primeira propriedade da estrutura *idt_gate_t* definida é o *low_offset* a qual será obtida a partir da função *low_16()*. De maneira análoga, *high_offset* será obtida a partir da função *high_16()*. Perceba que *low_offset* é definida como o primeiro espaço da estrutura. *high_offset* é definido como o segundo espaço da estrutura. Nesse caso, a ordem de todos esses elementos é importante. *sel* recebe o Kernel Segment Selector (valor 0x08). *always0*

refere-se ao “nível de execução” (*ring 0* refere-se ao kernel), o qual informa ao processador se uma certa interrupção pode ser executada apenas no *kernel*, em espaço de usuário ou em um espaço intermediário. O valor de `flags` é constante e será `0x8E` por padrão.

```
#include "idt.h"
#include "util.h"

void set_idt_gate(int n, uint32 handler) {
    idt[n].low_offset = low_16(handler);
    idt[n].sel = KERNEL_CS;
    idt[n].always0 = 0;
    idt[n].flags = 0x8E;
    idt[n].high_offset = high_16(handler);
}

void set_idt() {
    idt_reg.base = (uint32) &idt;
    idt_reg.limit = IDT_ENTRIES * sizeof(idt_gate_t) - 1;
    __asm__ __volatile__ ("lidt (%0)" : : "r" (&idt_reg));
}
```

Código 7: Especificação para *idt.c*.

A segunda estrutura presente no Código 6 `idt_register_t` também possui o atributo `__attribute__((packed))` para evitar otimizações ou quaisquer modificações impostas pelo compilador. Seu tamanho total exato deve ser de 48 bits. Os primeiros 16 bits são o “*limit*” os últimos 32 bits serão a “*base*”. A *base* irá receber o endereço do vetor `idt[IDT_ENTRIES]` (i.e. o endereço do início do vetor) e o tamanho de cada elemento é guardado no *limit*. Como esses são valores fixos para todos os elementos do vetor, cria-se apenas um único elemento chamado `idt_reg`.

Finalmente, temos as funções `set_idt_gate()` e `set_idt()` (Códigos 8 e 9). `set_idt_gate()` será chamada múltiplas vezes. Cada vez que for necessário ajustar o valor de um elemento interno ao vetor `idt[IDT_ENTRIES]`, chama-se essa função, passando-se como parâmetro o número *n* do elemento desejado e o *handler*, um inteiro interno de 32 bits o qual auxiliará a definir o conteúdo da estrutura. A função `set_idt()` é chamada uma única vez para repassar o vetor `idt[IDT_ENTRIES]` resultante para o processador. Assim, o valor de *base* será o endereço base do vetor (`&idt`) e *limit* receberá o tamanho do vetor. Após o preenchimento de *base* e *limit* chama-se a instrução *assembly* `lidt1` e passamos como parâmetro o endereço de `idt_reg` (`&idt_reg`) de modo que o processador possa lidar com a IDT.

O próximo passo é a especificação da Interrupt Service Routine (Rotina de Serviço de Interrupção ou ISR). Um exemplo de aplicação envolvendo ISR são as exceções (ex. quando

dividimos um número por zero). Quando não se implementam manipuladores de interrupção para certas interrupções (ex. divisão por zero), o SO vai reiniciar, geralmente, em um laço, por encontrar um programa para o qual não está preparado para resolver. A implementação parcial de *isr.h* está no Código 8.

```
#ifndef ISR_H
#define ISR_H

#include "types.h"

/* ISRs reservadas para exceções de CPU */

void isr0();

/* ... implementar até isr31()

string exception_messages[32];

void isr_install();

#endif
```

Código 8: Código para *isr.h*.

No Código 9, é apresentada a implementação da função `isr_install()`, a qual preenche o vetor `idt[IDT_ENTRIES]` com todos os endereços das funções de manipulação que precisam ser chamadas, sempre que uma interrupção é encontrada. No exemplo da divisão por 0, iremos chamar a `isr0()`. Se o processo encontrar a interrupção 1, chamará a `isr1()`, e assim por diante. As funções mencionadas são definidas no mesmo arquivo. Sempre que uma rotina `isrx()` for chamada, iremos imprimir uma exceção e então, parar a execução da CPU, mediante o uso da instrução *assembly* “`hlt`”.

O vetor de mensagens de exceção também é definido no arquivo *isr.c*. Este contém 32 mensagens, as quais são correspondentes às 32 ISRs implementadas (Código 9). A partir deste momento, o SO estará apto a lidar com qualquer tipo de interrupção, seja ela uma exceção, um clique do teclado, um mouse, um *clock tick* etc.

Pergunta-se: como você testará as rotinas de tratamento de interrupção (pelo menos uma delas)? Você deverá realizar mudanças em *build.sh* para que os novos arquivos “.c” sejam compilados e ligados ao seu kernel (`kernel.bin`). Além disso, crie um diretório `obj` separado e coloque seus arquivos objeto, gerados a partir das bibliotecas, nessa localidade. Também, adapte suas bibliotecas de modo a separar a implementação em um arquivo “.c”, conforme visto nos códigos fornecidos

nesta tarefa (ex. `idt.h` e `idt.c`).

```
#include "isr.h"
#include "idt.h"
#include "util.h"

void isr_install() {
    set_idt_gate(0, (uint32_t)isr0);
    // fazer até isr31
    //...
    set_idt(); //Carregado com ASM
}

/* Handlers */
void isr0()
{
    print(exception_messages[0]);
    asm("hlt");
}
/* ... */
/* fazer até isr31 */

/* Para imprimir as mensagens que definem cada exceção */

string exception_messages[] = {
    "Division by Zero",
    "Debug",
    "Non Maskable Interrupt",
    "Breakpoint",
    "Into Detected Overflow",
    "Out of Bounds",
    "Invalid Opcode",
    "No Coprocessor",
    "Double Fault",
    "Coprocessor Segment Overrun",
    "Bad TSS",
    "Segment Not Present",
    "Stack Fault",
    "General Protection Fault",
    "Page Fault",
    "Unknown Interrupt",
    "Coprocessor Fault",
    "Alignment Check",
    "Machine Check",
    "Reserved",
    // Completar demais posições do vetor com "Reserved"
};
```

Código 9: Código para `isr.c`.

TAREFA 3: MODIFICAÇÃO DA ESTRUTURA DO PROJETO E BUILD (06/11/2018 e 08/11/2018)

Nesta tarefa, modificaremos a estrutura do projeto e seu mecanismo de construção (*build*) para utilizarmos o *make*. Na raiz do seu projeto, local onde se localiza o script *build.sh*, você irá criar o arquivo Makefile, o qual ficará responsável (em consonância com a ferramenta make) por

gerenciar o processo de compilação e geração do kernel do SO.

make é uma ferramenta que permite a construção de um projeto de forma bastante “inteligente”. Inteligente aqui, refere-se ao fato de que, no caso do uso do *make*, se você compilar alguns arquivos e por ventura, atualizar um deles, apenas este será recompilado.

A organização do arquivo (Código 10) é bastante direta. No início do arquivo você tem a definição de diversas variáveis e seus valores (ex. COMPILER, LINKER, ASSEMBLER etc.). Para acessar o conteúdo da variável, basta usar a notação “\$(NOMEDAVARIAVEL)”. Em sua organização básica, um arquivo Makefile possui um alvo (*target*) que você deseja construir e este, por sua vez possui uma dependência. Por exemplo, na entrada “run: link”, run é o alvo e link é a dependência. A dependência pode ser um alvo em outro ponto do arquivo. Faça as modificações apropriadas em sua árvore de diretórios e na declaração dos arquivos de cabeçalho (.h), realizadas nos arquivos-fonte .c de modo que o arquivo Makefile fornecido funcione para o seu trabalho. Você não deve alterar o arquivo Makefile fornecido. Coloque informações suficientes em seu relatório de modo a identificar todas as modificações realizadas em seus códigos fonte/árvores de diretório para que o arquivo fornecido funcione. Além disso, demonstre o seu entendimento sobre o significado do arquivo fornecido no Código 10.

```

COMPILER = gcc
LINKER = ld
ASSEMBLER = nasm
CFLAGS = -m32 -c -ffreestanding
ASFLAGS = -f elf32
LDFLAGS = -m elf_i386 -T src/link.ld
EMULATOR = qemu-system-i386
EMULATOR_FLAGS = -kernel

SRCS = src/kernel.asm src/kernel.c src/idt.c src/isr.c src/kb.c src/screen.c
src/string.c src/system.c src/util.c
OBJS = obj/kasm.o obj/kc.o obj/idt.o obj/isr.o obj/kb.o obj/screen.o
obj/string.o obj/system.o obj/util.o
OUTPUT = B.OS/boot/kernel.bin

run: link
    $(EMULATOR) $(EMULATOR_FLAGS) $(OUTPUT)
    grub-mkrescue -o B.OS.iso B.OS/

link: compile $(OBJS)
    rm -rf B.OS
    mkdir B.OS
    mkdir B.OS/boot
    $(LINKER) $(LDFLAGS) -o $(OUTPUT) $(OBJS)

compile: $(SRCS)
    rm obj/ -rf
    mkdir obj/

obj/kasm.o: src/kernel.asm
    $(ASSEMBLER) $(ASFLAGS) -o obj/kasm.o src/kernel.asm

obj/kc.o: src/kernel.c
    $(COMPILER) $(CFLAGS) src/kernel.c -o obj/kc.o

obj/idt.o: src/idt.c
    $(COMPILER) $(CFLAGS) src/idt.c -o obj/idt.o

obj/kb.o: src/kb.c
    $(COMPILER) $(CFLAGS) src/kb.c -o obj/kb.o

obj/isr.o: src/isr.c
    $(COMPILER) $(CFLAGS) src/isr.c -o obj/isr.o

obj/screen.o: src/screen.c
    $(COMPILER) $(CFLAGS) src/screen.c -o obj/screen.o

obj/string.o: src/string.c
    $(COMPILER) $(CFLAGS) src/string.c -o obj/string.o

obj/system.o: src/system.c
    $(COMPILER) $(CFLAGS) src/system.c -o obj/system.o

obj/util.o: src/util.c
    $(COMPILER) $(CFLAGS) src/util.c -o obj/util.o

build: all
    # Activate the install xorrr if you do not have it already installed
    # sudo apt install xorriso
    rm B.OS/boot/grub/ -rf
    mkdir B.OS/boot/grub
    echo set default=0 >> B.OS/boot/grub/grub.cfg
    echo set timeout=0 >> B.OS/boot/grub/grub.cfg

```

```

echo menuentry "B.OS" { >> B.OS/boot/grub/grub.cfg
echo set root='(hd96)' >> B.OS/boot/grub/grub.cfg
echo multiboot /boot/kernel.bin >> B.OS/boot/grub/grub.cfg
echo } >> B.OS/boot/grub/grub.cfg

clean:
rm -rf obj/
rm -rf B.OS/
rm B.OS.iso

```

Código 10: Código para o Makefile.

TAREFA 4: CONSTRUÇÃO DE UM SHELL SIMPLES** (13, 15 (extra-classe), 20 e 22/11/2018)

Para a criação do *shell*, esta tarefa utiliza as últimas modificações nas bibliotecas desenvolvidas até o momento e introduz novas modificações. Para esta prática, iremos criar um arquivo de biblioteca e sua contraparte *shell.h* e *shell.c*. Além disso, faça as seguintes modificações indicadas no Código 11 em *kernel.c*. Por ser muito simples, o arquivo de biblioteca para *shell.h* fica a cargo do aluno. O arquivo para *shell.c* é apresentado no Código 12. Perceba neste código que introduzimos a função `malloc()`. Esta será inserida no contexto do arquivo *util.h* e *util.c*. Além disso, também foram inseridas modificações em funções pertencentes a *screen.h* e *screen.c*. (Código 13). Fica a cargo do aluno introduzir todas as modificações necessárias nos arquivos “*.h” referentes às funções introduzidas nas bibliotecas, conforme apresentado. Além disso, também será necessário fazer alguma modificação no arquivo *Makefile*, de modo a implementar o suporte a *shell.h* e *shell.c* (Código 14).

```

//...
#include "../include/shell.h"

int kmain()
{
    isr_install();
    clearScreen();

    //...

    print("Bem vindo ao B.OS ----->\n\n\n\nEntre um
comando\n");

    launch_shell(0);
}

```

Código 11: Modificações a serem inseridas em *kernel.c*.

```

#include "../include/util.h"

// ...

/* NOVAS funcoes */

string int_to_string(int n) {
    string ch = malloc(50);
    int_to_ascii(n, ch);
    int len = strlen(ch);
    int i = 0,
        j = len - 1;
    while (i < (len/2 + len%2)) {
        char tmp = ch[i];
        ch[i] = ch[j];
        ch[j] = tmp;
        i++;
        j--;
    }
    return ch;
}

int str_to_int(string ch) {
    int n = 0;
    int p = 1;
    int strlen = strlen(ch);
    int i;

    for(i = strlen-1; i >= 0; i--) {
        n += ((int)(ch[i] - '0')) * p;
        p *= 10;
    }
    return n;
}

void * malloc(int nbytes) { // maneira simples mas
    }
                                // malloc - nao faz uso de
                                // mcb

    char variable[nbytes];
    return &variable;
}

```

Código 12: Modificações a serem inseridas em *util.c*.

Algumas observações são válidas: o processamento do *backspace* deverá ser adequado de modo a lidar com o “*prompt*” do *shell*. Se o usuário começar a dar *backspace* indiscriminadamente, acabará apagando o *prompt*. Além disso, ao digitar um comando, se o *backspace* for pressionado, isso irá influenciar no tratamento do comando (ex. digitando-se *clear*, pressionando-se o *backspace* e novamente o ‘r’, não resultará no *clear*). Assim sendo, tratem esse *caveat*.

```

#include "../include/screen.h"

// ...

int color = 0x0F; // NOVO: armazena a cor corrente
                  // usada no terminal

// NOVO: funcao clearLine() modificada para usar color
void clearLine(uint8 from, uint8 to)
{
    //...

    for (i; i < (sw * to * sd); i++) // NOVO
    {
        vidmem[(i/2)*2+1] = color; // NOVO: referente a cor
        vidmem[(i/2)*2] = 0; // NOVO: referente ao caractere
    }
}

// ...

/* NOVAS funcoes */
void set_screen_color(int text_color, int bg_color) {
    color = (bg_color << 4) | text_color;
}

void set_screen_color_from_color_code(int color_code) {
    color = color_code;
}

void print_colored(string ch, int text_color, int
bg_color) {
    int current_color = color;
    set_screen_color(text_color, bg_color);
    print(ch);
    set_screen_color_from_color_code(current_color);
}

```

Código 13: Modificações a serem inseridas em *screen.c*.

```

//...
#include "../include/shell.h"

void launch_shell(int n) {
    string ch = (string) malloc(200); //definido em util.h
    do {
        print("B.OS (");
        print(int_to_string(n));
        print(">");
        ch = readStr(); // memory_copy(readStr(), ch, 100);
        if(strEql(ch, "cmd")) {
            print("\nVoce ja esta no cmd. Novo shell recursivo
aberto\n");
            launch_shell(n+1);
        }
        else if (strEql(ch, "clear"))
            clearScreen();
        else if (strEql(ch, "soma"))
            sum();
        else if (strEql(ch, "exit"))
            print("\nAte logo!\n");
        else if (strEql(ch, "echo"))
            echo();
        else if (strEql(ch, "sort"))
            sort();
        else if (strEql(ch, "fibo"))
            fibonacci();
        else if (strEql(ch, "mmc"))
            gcd();
        else if (strEql(ch, "help"))
            help();
        else if (strEql(ch, "cor"))
            set_bg_color();
        else {
            print("\nComando desconhecido\n");
        }
    } while(!strEql(ch, "exit"));
}

void sum() {
    int n, i, s;

    print("\nQuantos n.os: ");
    n = str_to_int(readStr());
    i = 0;
    print("\n");
    int arr[n];
    fill_array(arr, n);
    s = sum_array(arr, n);
    print("Resultado: ");
    print(int_to_string(s));
    print("\n");
}

void fill_array(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        print("ARR[");
        print(int_to_string(i));
        print("]:");
        arr[i] = str_to_int(readStr());
        print("\n");
    }
}

```

```

void print_array(int arr[], int n) {
    int i;

    for (i = 0; i < n; i++) {
        print(int_to_string(arr[i]));
        print(" ");
    }
    print("\n");
}

int sum_array(int arr[], int n) {
    int i;
    int s = 0;

    for (i = 0; i < n; i++) {
        s += arr[i];
    }
    return s;
}

void echo() {
    print("\n");
    string str = readStr();
    print("\n");
    print(str);
    print("\n");
}

void sort() {
    int arr[100];
    print("\nTamanho do vetor: ");
    int n = str_to_int(readStr());
    print("\n");
    fill_array(arr, n);
    print("Antes da ordenacao:\n");
    print_array(arr, n);
    print("\nOrdem: (1 - crescente/ 0 - decrescente): ");
    int order = str_to_int(readStr());
    insertion_sort(arr, n, order);
    print("Apos ordenacao:\n");
    print_array(arr, n);
}

void insertion_sort(int arr[], int n, int order) { // 1 -
crescente e 0 - decrescente
    int i;

    for (i = 0; i < n; i++) {
        int aux = arr[i];
        int j = i;

        while((j > 0) && ((aux < arr[j-1]) && order)) {
            arr[j] = arr[j-1];
            j = j-1;
        }
        arr[j] = aux;
    }
}

void fibonacci() {
    int i, n;

```



```

    print("\nQuantos elementos: ");
    n = str_to_int(readStr());
    print("\n");
    for (i = 0; i < n; i++) {
        print("Fibo ");
        print(int_to_string(i));
        print(" : ");
        print(int_to_string(fibo(i)));
        print("\n");
    }
}

int fibo(int n) {
    if (n < 2)
        return 1;
    else
        return fibo(n-1) + fibo (n-2);
}

void set_bg_color() {
    print("\nCodigos de cores: ");
    print("\n0 : preto");
    print_colored("\n1 : azul", 1,0);
    print_colored("\n2 : verde", 2,0);
    print_colored("\n3 : ciano", 3,0);
    print_colored("\n4 : vermelho", 4,0);
    print_colored("\n5 : purpura", 5,0);
    print_colored("\n6 : laranja", 6,0);
    print_colored("\n7 : cinza", 7,0);
    print_colored("\n8 : cinza escuro", 8,0);
    print_colored("\n9 : azul claro", 9,0);
    print_colored("\n10 : verde claro", 10,0);
    print_colored("\n11 : azul mais claro", 11,0);
    print_colored("\n12 : vermelho claro", 12,0);
    print_colored("\n13 : rosa", 13,0);
    print_colored("\n14 : amarelo", 14,0);
    print_colored("\n15 : branco", 15,0);

    print("\n\n Cor do texto: ");
    int text_color = str_to_int(readStr());
    print ("\n\n Cor do fundo: ");
    int bg_color = str_to_int(readStr());
    set_screen_color(text_color, bg_color);
    clearScreen();
}

void help() {
    print("\ncmd\t: executa um novo shell recursivo");
    print("\nclear\t: limpa a tela");
    print("\nsoma\t: computa a soma de n numeros");
    print("\necho\t: imprime um dado texto");
    print("\nsort\t: ordena n numeros");
    print("\nfibo\t: imprime os n primeiros numeros da
serie de Fibonacci");
    print("\ncor\t: muda a cor do terminal");
    print("\nquit\t: sai do shell corrente");

    print("\n\n");
}

```

Código 14: Modificações a serem inseridas em *shell.c*.